Why Frontend?

What it is, why it matters, and what is worth learning when AI can write code.

WARM-UP QUIZ

What do you remember?

Scenarios

Python: output or error?

Output or error?

Fest sign-ups arrive from a web form, and some students submit twice.

```
signups = ["asha", "ravi", "asha", "meera", "ravi"]
unique = set(signups)
print(len(signups), len(unique))
print("asha" in unique)
```

ANSWER · DSA

It prints two lines

```
5 3
True
```

Why

A set keeps one copy of each value, so 5 sign-ups become 3 students. Checking “in” on a set is a fast lookup, like a hash map.

Full-stack link

Cleaning duplicates in code is a patch. The database can refuse duplicates in the first place. Watch for that later in the quiz.

The canteen app says “ Masala dosa: available” . You reach the counter and it’ s sold out. Which change actually fixes this?

A Refresh the menu screen every second

B Keep stock in the database; the server refuses orders once it hits zero

C Save the stock count on each student’s phone

D Write “may be sold out” under every dish

ANSWER · DB

B · Keep stock in the database; the server refuses orders once it hits zero

Why not the others

A loads the server and is still out of date between refreshes. **C** gives every phone its own wrong copy. **D** hides the problem.

Full-stack point

One source of truth. The frontend **shows**, the backend **decides**, the database **remembers**.

Output or error?

A student copies their cart into an order, then adds one more item to the order.

```
cart = ["dosa", "idli"]
order = cart
order.append("vada")
print(cart)
```

ANSWER · DSA

The cart changed too

```
['dosa', 'idli', 'vada']
```

Why

order = cart doesn't copy the list. Both names point to the same list. For a real copy: order = cart.copy()

Full-stack link

JavaScript works the same way. In React, changing shared data by accident is a classic bug, which is why you'll learn to make copies.

The PRD says: “Students can register for the fest.” Before building the screen, which question matters most?

A Which database should we use?

B Which font should the heading use?

C What should students see if the event is full, or registration has closed?

D Can we use the newest framework?

ANSWER · PRD

C · What students see when the event is full, or registration has closed

Why

The PRD describes only the happy path. Every screen also needs its other states: full, closed, already registered, no network.

Full-stack point

The backend must report each state, and the frontend must show it. If the PRD is silent, the frontend developer is usually the first to notice.

Output or error?

The server receives a quantity from the canteen order form.

```
qty = "2"    # every form field arrives as text
price = 40
print(qty * 3)
print(qty + price)
```

ANSWER · PYTHON

One line prints, one line crashes

```
222
Traceback (most recent call last): … line 4
TypeError: can only concatenate str (not "int") to str
```

Why

“2” is text. Text times 3 repeats it. Text plus a number is an error. Fix: `int(qty) + price`

Full-stack link

Data from forms and APIs arrives as text. Convert and validate it on the server: what if someone types “two” or -5?

You change your phone number on the profile screen and it shows the new one. After you log out and back in, the old number is back. What most likely went wrong?

A The database ran out of space

B The screen updated, but the save never reached the database

C The browser loaded an old font

D The number was too long to display

ANSWER · DB

B · The screen updated, but the save never reached the database

Why

The request to the server failed or was never sent. After login, the app reads from the database, which still has the old number.

Full-stack point

What’s on screen is not proof it was saved. A good frontend waits for the server’s reply, and says clearly when saving failed.

Output or error?

The canteen server looks up how many of each dish are left.

```
stock = {"dosa": 3, "idli": 0}
print(stock.get("idli", 0))
print(stock["vada"])
```

ANSWER · DSA

0, then a KeyError

```
0
Traceback (most recent call last): … line 3
KeyError: 'vada'
```

Why

.get() returns a default when a key is missing. Square brackets crash instead.

Full-stack link

A request for a dish that doesn’t exist shouldn’t crash the server. It should reply “not found”, and the screen should say so kindly.

The fest app lists 2,000 events. Students search by name and filter by day, mostly on budget phones. What's the smartest plan?

A Send all 2,000 to the phone and loop through them on every keystroke

B The server asks the database for matches and sends back the first 20

C Download all 2,000 again on every keystroke

D Re-sort all 2,000 on the phone on every keystroke

ANSWER · DSA

B · The server asks the database for matches and sends back the first 20

Why

Databases are built to search big tables fast. Sending 20 results instead of 2,000 saves the student's data and battery. This is called pagination.

Full-stack point

Deciding where the work happens is a full-stack choice. For a list of 50 items, filtering on the phone is perfectly fine.

Output or error?

The attendance-alerts feature: who is below 75%, and who has the best attendance?

```
students = [
    {"name": "Asha", "att": 82},
    {"name": "Ravi", "att": 68},
    {"name": "Meera", "att": 91},
]
low = [s["name"] for s in students if s["att"] < 75]
top = max(students, key=lambda s: s["att"])
print(low, top["name"])
```

ANSWER · DSA

A list, then a name

```
['Ravi'] Meera
```

Why

low keeps only names below 75, so it is a list. max with a key compares attendance and returns the whole student; we print just the name.

Full-stack link

A list of dictionaries is exactly what an API sends to a frontend. You'll filter lists like this in JavaScript on Day 1.

A new PRD requirement: “A student can’t register twice for the same event.” Where must this rule be enforced?

A Only by hiding the Register button after one click

B On the server and in the database, even if the screen also hides the button

C Writing it in the PRD is enough

D Nowhere: students won’t try it

ANSWER · PRD

B · On the server and in the database, even if the screen also hides the button

Why

Hiding a button isn’t protection: people double-tap on slow networks, open two tabs, or send requests without the app at all.

Full-stack point

Frontend rules are for convenience. Backend rules are for correctness. The PRD states the rule; the team decides where it lives.

Output or error?

A real (tiny) database, built into Python. Two students try to register with the same roll number.

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE students (roll TEXT PRIMARY KEY, name TEXT)")
db.execute("INSERT INTO students VALUES ('24CS101', 'Asha')")
print("first saved")
db.execute("INSERT INTO students VALUES ('24CS101', 'Ravi')")
print("second saved")
```

ANSWER · DB

The database says no

```
first saved
Traceback (most recent call last): … line 6
sqlite3.IntegrityError: UNIQUE constraint failed: students.roll
```

Why

A primary key must be unique, so the database refuses the second row. “second saved” never prints.

Full-stack link

This is question 10’s answer in action. The server catches this error and replies “already registered”, and the screen shows a friendly message.

At lunch, 500 students order through the canteen app. The kitchen's screen must show orders in the exact order they were placed. Which structure fits?

A Stack

B Queue

C Hash map

D Tree

ANSWER · DSA

B · A queue: first in, first out

Why

The oldest order is cooked first. A stack would serve the newest first, and the earliest students would wait forever.

Full-stack point

The student’s app and the kitchen screen are two different frontends reading the same orders from one backend and database.

Output or error?

The kitchen’s order line. Careful: two different ways to take an order out.

```
from collections import deque
orders = deque()
orders.append("#101")
orders.append("#102")
orders.append("#103")
print(orders.popleft(), orders.pop())
print(list(orders))
```

ANSWER · DSA

Oldest from the front, newest from the back

```
#101 #103
['#102']
```

Why

popleft() takes the oldest (queue). pop() takes the newest (stack). Only #102 is left.

Full-stack link

Using pop() by mistake would cook the newest order first. One wrong method name, and real students wait.

Output or error?

The server sends a student’s profile to the app as JSON text.

```
import json
reply = '{"name": "Asha", "dept": "CSE", "attendance": 82}'
data = json.loads(reply)
print(data["attendance"] + 5)
print(reply["attendance"])
```

ANSWER · PYTHON

87, then a TypeError

```
87
Traceback (most recent call last): … line 5
TypeError: string indices must be integers, not 'str'
```

Why

reply is just text. json.loads turns it into a dictionary, data, which you can look up by key.

Full-stack link

Every API reply arrives as text. In JavaScript you'll call res.json() to do the same job, on Day 3.

REFRESHER · DB

A database is where an app’s data lives

Tables of rows and columns. Each row has a key that never repeats. Queries ask for exactly the rows you need.

Frontend link: every screen shows or changes data like this, through the backend.

students

roll_no (key)	name	dept
24CS101	Asha Rao	CSE
24CS102	Ravi Kumar	CSE
24EC045	Asha Rao	ECE
24ME012	Meera Iyer	MECH

Two students called Asha Rao, but different roll numbers. That is why the key matters.

REFRESHER · PRD

A PRD says what to build, for whom, and why

It leaves the how to the team. Every requirement is something you can test.

Frontend link: you turn requirements into screens. A vague PRD gives a confusing app, and a vague AI prompt gives a generic one.

PRD · ATTENDANCE ALERTS

Problem: students find out too late that their attendance is below 75%.

Users: undergraduate students, mostly on phones.

Requirement: a student sees a warning on the home screen when a subject drops below 80%.

Success: fewer students below 75% at the end of term.

Out of scope: emails to parents.

REFRESHER · DSA

DSA is how code organises data so it stays fast

Array

Items in order: a todo list, a feed of posts

Hash map

Instant lookup: find a user by their ID

Stack

Last in, first out: Undo, and the Back button

Queue

First in, first out: uploads, messages to send

Tree

Things inside things: every web page is one

Frontend link: these aren’t just exam topics. You’ll use arrays and maps every day, and the page itself is a tree.

PUTTING IT TOGETHER

One feature: “Show me this week’s timetable”

PRD

A student sees today’s classes as soon as they log in.

DB

A classes table: day, time, subject, room.

DSA

An array of classes, grouped by day, sorted by time.

FRONTEND

Today first, next class highlighted, readable on a phone.

Everything you already know ends up somewhere. Frontend is where it finally meets the user.